

Home Lab Project Week5, Day1

Task:

Prepare the Kali Linux and Ubuntu VMs for controlled FTP/HTTP traffic generation. Verify connectivity and identify the appropriate services on the target.

Given the UTM VirtualBox home lab with Kali Linux as the testing system and Ubuntu as the protected/target system, this task is essentially preparing the two VMs so I can later generate and analyze controlled FTP and HTTP traffic.

Lab addresses are:

- **Kali Linux:** 192.168.64.2
- **Ubuntu:** 192.168.64.3
- **Network:** 192.168.64.0/24

1. Verify connectivity from Kali

On Kali, open Terminal and type:

ip addr

Confirm Kali has:192.168.64.2

```
(kali@kali)-[~]
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether ea:f0:2f:00:11:3a brd ff:ff:ff:ff:ff:ff
    inet 192.168.64.2/24 brd 192.168.64.255 scope global dynamic noprefixroute
        valid_lft 2222sec preferred_lft 2222sec
    inet6 fda6:6bc3:cab9:adbc:1403:4cbc:c0a3:bcc6/64 scope global temporary dynamic
        valid_lft 529624sec preferred_lft 10973sec
    inet6 fda6:6bc3:cab9:adbc:e8f0:2fff:fe00:113a/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 2591978sec preferred_lft 604778sec
    inet6 fe80::e8f0:2fff:fe00:113a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

And Ubuntu has:

192.168.64.3

```
ubuntu@ubuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 1a:63:23:39:a4:2c brd ff:ff:ff:ff:ff:ff
    inet 192.168.64.3/24 metric 100 brd 192.168.64.255 scope global dynamic enp0s1
        valid_lft 2714sec preferred_lft 2714sec
    inet6 fda6:6bc3:cab9:adbc:1863:23ff:fe39:a42c/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 2591928sec preferred_lft 604728sec
    inet6 fe80::1863:23ff:fe39:a42c/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu:~$
```

Then, let's ping Ubuntu:

ping -c 4 192.168.3

We got a reply ,64 bytes from 192.168.64.3with zero packet loss. Because we got replies, the basic VM-to-VM connection is working.

```
(kali㉿kali)-[~]
└─$ ping -c 4 192.168.64.3
PING 192.168.64.3 (192.168.64.3) 56(84) bytes of data:
64 bytes from 192.168.64.3: icmp_seq=1 ttl=64 time=1.25 ms
64 bytes from 192.168.64.3: icmp_seq=2 ttl=64 time=1.10 ms
64 bytes from 192.168.64.3: icmp_seq=3 ttl=64 time=1.08 ms
64 bytes from 192.168.64.3: icmp_seq=4 ttl=64 time=1.06 ms

— 192.168.64.3 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3016ms
rtt min/avg/max/mdev = 1.056/1.120/1.252/0.077 ms
```

2. Identify services running on Ubuntu

From **Kali**, we run:

nmap -sV 192.168.64.3

This identifies open ports and attempts to determine the services running on Ubuntu.

For our task, we are particularly looking for open ports

We might see something like: PORT STATE SERVICE VERSION

21/tcp open ftp

80/tcp open http

```
(kali㉿kali)-[~]  
$ nmap -sV 192.168.64.3  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-16 09:00 PDT  
Note: Host seems down. If it is really up, but blocking our ping probes, try  
-Pn  
Nmap done: 1 IP address (0 hosts up) scanned in 3.13 seconds
```

In the response we got, the host is either down or blocking the ping probes.

To identify the services running on the target Ubuntu machine, I need to bypass Nmap's initial "ping check" since the host's firewall is blocking those specific probes.

Because the machine responded to your initial ICMP ping, it is definitely online. I can force Nmap to skip host discovery and proceed directly to scanning ports and identifying services by adding the **-Pn flag**.

Here are the best commands to achieve this, ranging from a quick scan to a thorough investigation:

Let's run the command:

```
nmap -Pn -sV 192.168.64.3
```

This command skips the host-down check, scans the top 1,000 most common ports, and attempts to determine what services and version numbers are running.

```
(kali㉿kali)-[~]  
$ nmap -Pn -sV 192.168.64.3  
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-16 09:32 PDT  
Nmap scan report for 192.168.64.3  
Host is up (0.058s latency).  
All 1000 scanned ports on 192.168.64.3 are in ignored states.  
Not shown: 990 filtered tcp ports (no-response), 10 filtered tcp ports (host-unreach)  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 31.49 seconds
```

The results of the command show that **the target host is up, but all 1,000 scanned TCP ports are blocked by a firewall or network security control.**

Specifically, Nmap breaks down the **ignored states** of the top 1,000 ports as follows:

- **990 filtered TCP ports (no-response):** The firewall dropped these packets silently, causing the connection to time out.

- **10 filtered TCP ports (host-unreach):** The network router or firewall explicitly replied with an ICMP "Destination Host Unreachable" error when Nmap tried to probe them.

```
(kali㉿kali)-[~]
└─$ nmap -Pn -p- -sV 192.168.64.3
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-16 09:44 PDT
Nmap scan report for 192.168.64.3
Host is up (0.055s latency).
All 65535 scanned ports on 192.168.64.3 are in ignored states.
Not shown: 65525 filtered tcp ports (no-response), 10 filtered tcp ports (host-unreach)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 183.05 seconds
```

What This Means

The `-Pn` flag successfully forced Nmap to scan the target, but a tight firewall configuration (like `ufw` or `iptables` on the Ubuntu machine) is proactively dropping or rejecting standard scanning traffic. No open services were found within Nmap's default top 1,000 ports.

How to Proceed

Since the standard 1,000 ports are locked down, the services on this Ubuntu machine might be running on non-standard ports. I can try these next steps to find them:

Scan All Ports: Run a full scan across every possible port to see if a service is hidden high up (e.g., port 8080, 9000, etc.):

bash

```
nmap -Pn -p- --min-rate 1000 192.168.64.3
```

The results of my full port scan confirm that **all 65,535 TCP ports on the Ubuntu target are completely filtered and blocked by a firewall.**

Specifically:

- **65,525 ports** silently dropped the connection (`no-response`).
- **10 ports** explicitly rejected the connection (`host-unreach`).

Because absolutely zero TCP ports are responding, standard TCP-based network scanning cannot reveal the services running on this machine. This implies that the

Ubuntu target has an extremely strict firewall rule (like a default drop/deny policy on `ufw` or `iptables`) or it is dynamically blocking Kali's IP address due to port scanning behavior (rate-limiting / knocking).

3. Check Ubuntu's services

On **Ubuntu**, let's run:

`sudo ss -tulpn`

```
ubuntu@ubuntu:~$ sudo ss -tulpn
[sudo] password for ubuntu:
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port
Process
udp UNCONN 0 0 0.0.0.0:5353 0.0.0.0:*
users:((("avahi-daemon",pid=741,fd=12))
udp UNCONN 0 0 192.168.64.3%enp0s1:68 0.0.0.0:*
users:((("systemd-network",pid=626,fd=18))
udp UNCONN 0 0 0.0.0.0:55806 0.0.0.0:*
users:((("avahi-daemon",pid=741,fd=14))
udp UNCONN 0 0 [::]:33788 [::]:*
users:((("avahi-daemon",pid=741,fd=15))
udp UNCONN 0 0 [::]:5353 [::]:*
users:((("avahi-daemon",pid=741,fd=13))
tcp LISTEN 0 128 0.0.0.0:22 0.0.0.0:*
users:((("sshd",pid=934,fd=3))
tcp LISTEN 0 128 127.0.0.1:631 0.0.0.0:*
users:((("cupsd",pid=813,fd=7))
tcp LISTEN 0 128 [::]:631 [::]:*
users:((("cupsd",pid=813,fd=6))
tcp LISTEN 0 128 [::]:22 [::]:*
users:((("sshd",pid=934,fd=4))
```

Now that I have run `sudo ss -tulpn` directly on the Ubuntu target, I can see exactly what services are listening locally.

The command reveals that **SSH (port 22) is actively listening on TCP**.

Why Nmap Failed ?

Explanation:

The only real remote target is **SSH on port 22**. Because `sshd` is listening on `0.0.0.0:22`, our Kali machine *should* be able to talk to it. But, the fact that Nmap showed port 22 as "filtered" means an **internal software firewall (like UFW or iptables)** on this Ubuntu machine is explicitly blocking incoming connections on port 22 from your network or from your specific Kali IP address.

Issue: at one point Kali could not communicate with the Ubuntu target. Initially, even ICMP pings failed; after networking was corrected, service connections still timed out.

Root cause (two issues, resolved in sequence):

1. **No shared network between the VMs.** With the original single-NAT configuration, each VM sat behind its own virtual NAT and had no common network, so no traffic — including pings — could pass between them.
2. **Host firewall dropping traffic.** Once the VMs shared a network, Ubuntu's firewall (ufw) silently dropped incoming packets from Kali. Because ufw drops rather than rejects, the failure appeared as a timeout rather than a refusal.

Fixes applied:

Stage 1 — networking: Configured two adapters on each VM in Oracle VirtualBox:

- Adapter 1 — NAT (internet access for updates/installs)
- Adapter 2 — Host-Only (private VM-to-VM network)

Both VMs then received addresses on the host-only network: Kali `192.168.78.3`, Ubuntu `192.168.78.5`. Pings between the VMs now succeeded.

Stage 2 — firewall: With networking confirmed, allowed the lab subnet through Ubuntu's firewall:

```
bash
sudo ufw status          # confirmed active and dropping traffic
sudo ufw allow from 192.168.78.0/24  # allowed the host-only lab network
```

Result: Kali and Ubuntu communicate fully — both ICMP and service traffic.

Verification:

```
bash
ping -c 4 192.168.78.5  # confirms Stage 1 (network)
nmap -p 21 192.168.78.5  # confirms Stage 2 (service reachable, not filtered)
```

Since the ping comes back with a reply and zero packets loss, then the network is fine but Ubuntu is dropping or ignoring port 21. On Kali terminal, running nmap shows that the port state on Ubuntu is filtered.

```
(kali㉿kali)-[~]  
$ nmap -p 21 192.168.78.5  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-20 08:35 -0400  
Nmap scan report for 192.168.78.5  
Host is up (0.0012s latency).  
  
PORT      STATE      SERVICE  
21/tcp    filtered  ftp  
MAC Address: 08:00:27:E3:80:BC (Oracle VirtualBox virtual NIC)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.86 seconds
```

21/tcp filtered means packets to port 21 are being silently dropped on Ubuntu, and a firewall is by far the most likely cause.

Let's verify that the Ubuntu interface state is up.

```
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:e3:80:bc brd ff:ff:ff:ff:ff:ff  
    altname enx080027e380bc  
    inet 192.168.78.5/24 brd 192.168.78.255 scope global dynamic noprefixroute enp0s8  
        valid_lft 377sec preferred_lft 377sec  
    inet6 fe80::27ff:7077:590b:3254/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever
```

Let's check the firewall on ubuntu with the command is:

sudo ufw status

```
vboxuser@Ubuntu:~$ sudo ufw status  
[sudo: authenticate] Password:  
Status: active  
vboxuser@Ubuntu:~$ sudo ufw allow 21/tcp  
Rule added  
Rule added (v6)  
vboxuser@Ubuntu:~$
```

When applying the command on Ubuntu terminal: sudo ss -tlnp | grep :

If ufw says active, and it does. It's likely the culprit. It drops packets silently, which shows up as a timeout, and it usually still answers pings, so a working ping doesn't rule it out. Either allow the lab subnet or switch it off for the lab:

sudo ufw allow from 192.168.78.0/24

```
vboxuser@Ubuntu:~$ sudo ufw status
[sudo: authenticate] Password:
Status: active
vboxuser@Ubuntu:~$ sudo ufw allow 21/tcp
Rule added
Rule added (v6)
vboxuser@Ubuntu:~$ sudo ufw disable
Firewall stopped and disabled on system startup
vboxuser@Ubuntu:~$
```

For the Wireshark capture, Kali now has two interfaces, so let's pick **eth1**, the NAT adapter and it will show this traffic. Use the capture filter **host 192.168.78.5 and port 21**.

That rules out the VirtualBox networking and Kali's routing, so the problem is specifically port 21 on Ubuntu. Probing the port from Kali will show which Ubuntu-side problem it is:

```
nmap -p 21 192.168.78.5
```

After running nmap, we found out about the port's closed state.

```
(kali@kali)-[~]
$ nmap -p 21 192.168.78.5
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-20 08:50 -0400
Nmap scan report for 192.168.78.5
Host is up (0.0038s latency).

PORT      STATE SERVICE
21/tcp    closed ftp
MAC Address: 08:00:27:E3:80:BC (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.69 seconds
```

Closed means progress, it means Ubuntu is answering but nothing is listening on port 21, so no FTP server is running.

```
(kali@kali)-[~]
$ nmap -sV -p 21 192.168.78.5
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-20 21:52 -0400
Nmap scan report for 192.168.78.5
Host is up (0.0014s latency).

PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
MAC Address: 08:00:27:E3:80:BC (Oracle VirtualBox virtual NIC)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 1.15 seconds
```

Now that the port is open, let's start wireshark and start capturing traffic.

To start an ftp session we need to update the system's package list, install a secure FTP server.

On Ubuntu, install and start vsftpd

```
sudo apt update && sudo apt install vsftpd -y
```

```
sudo systemctl enable --now vsftpd
```

```
sudo ss -tlnp | grep :21
```

```
vboxuser@Ubuntu:~$ sudo ss -tlnp | grep :21
[sudo: authenticate] Password:
LISTEN 0      32          *:21        *:~
vboxuser@Ubuntu:~$
```

"Connection timed out" (rather than "connection refused") is actually a useful clue — it means the SYN packet on port 21 got no response at all. That points to either the network path not being up yet, or something on Ubuntu silently dropping the packet — as opposed to Ubuntu being reachable but nothing listening on port 21, which would say "refused" instead.

Then type **bye** to leave the **ftp>** prompt on Kali and retry **ftp 192.168.78.5**.

filtered means something on Ubuntu is silently dropping the packets, which matches the timeout. It's almost always the firewall.

Once nmap shows **open**, start Wireshark on **eth1** with the capture filter **host 192.168.78.5 and port 21**, then connect:

```
ftp 192.168.78.5
```

Log in with Ubuntu's own account (**vboxuser** and its password). The Name and Password prompts should appear this time, and the credentials will show up in Follow → TCP Stream.

If the nmap result doesn't match any of these, send me the output and I'll narrow it down.

Re-scan from Kali

```
nmap -p 21 192.168.78.5
```

closed means the firewall is out of the way but no FTP server is running yet, so go to step 3.
open means you can skip to step 4.

The last command should show a listener on **0.0.0.0:21** or ***:21**. Re-run the nmap scan and it should now say **open**. The default Ubuntu config allows local user logins, so no config changes are needed. Since the password will be visible in the capture, I need to come up with a throwaway account for the lab instead of your real one:

```
sudo adduser labuser
```

Run the capture

Start wireshark and start capturing.

If HTTP is needed

For a simple controlled HTTP test, Ubuntu can run Apache.

On Ubuntu:

```
sudo apt update
```

```
sudo apt install apache2
```

Start it:

```
sudo systemctl enable --now apache2
```

Check:

```
sudo systemctl status apache2
```

Then from Kali:

```
curl http://192.168.64.5
```

If Apache is working, Kali should receive the Ubuntu web page HTML.

You can also open Firefox on Kali and visit:

```
http://192.168.64.5
```

Prepare Wireshark on Kali

This is an important part of my cybersecurity home lab. Wireshark will help in capturing the traffic.

On Kali, main menu, type on the search bar.

Select the network interface connected to your **192.168.64.0/24 isolated network**.

Start the capture.

For HTTP traffic, I can use the filter: http

For FTP, I can use the filter: ftp

I can also use:

ip.addr == 192.168.64.3 to focus on traffic involving Ubuntu.

Generate controlled traffic

HTTP

From Kali:

```
curl http://192.168.64.3
```

Run it a few times:

```
curl http://192.168.64.3
```

Wireshark should capture the communication between:

```
Kali 192.168.64.2
  ↓
Ubuntu 192.168.64.3
  TCP/80
```

FTP

Once FTP is configured, from Kali you can test the connection:

```
ftp 192.168.64.3
```

```
(kali@kali)~$ ftp 192.168.78.5
Connected to 192.168.78.5.
220 (vsFTPD 3.0.5)
Name (192.168.78.5:kali):
```

This generates FTP connection traffic that you can observe in Wireshark.

8. The documentation for this task:

Kali Linux

- IP: 192.168.64.2
- Role: Testing/traffic-generation system
- Tools: Nmap, Wireshark, curl, FTP client

Ubuntu

- IP: 192.168.64.3
- Role: Protected/target system
- Services: HTTP and/or FTP
- HTTP: TCP 80
- FTP: TCP 21

Connectivity verification:

```
ping -c 4 192.168.64.3
```

Service identification:

```
nmap -sV 192.168.64.3
```

HTTP test:

```
curl http://192.168.64.3
```

FTP test:

```
ftp 192.168.64.3
```

Wireshark → capture traffic → filter HTTP/FTP

Important Observation:

Because my lab is an **isolated 192.168.78.0/24 network**, I keep the FTP/HTTP testing inside these VMs, and there is no need to generate traffic against anything outside the lab.